

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0505

FACULTY NAME :Ms.K.Pavithra

Name	M.Thanuja
Register Number	192425356

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Constraint-Based Problem Solving (High)

Assessment Tool Weightage:40%

Q. No	Question
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with Prepared Statement.
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.

Code of Conduct:

M.THANUJA(192425356)_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:M.Thanuja

CO5 ASSESSMENT TOOL 1

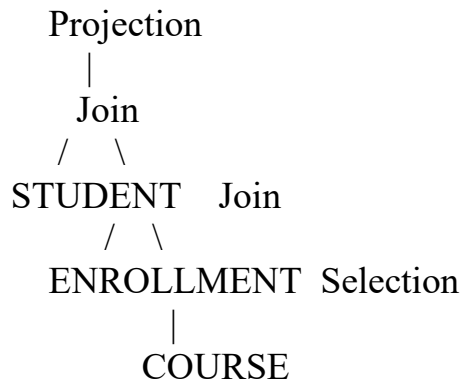
RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application ; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

S.No	Problem Understanding (25%) /1.25 Marks	Constraint Analysis (25%) /1.25 Marks	Solution Development (25%) /1.25 Marks	Application of Concepts (15%) /0.75 Marks	Justification (10%) /0.50 Marks	Total /5 Marks
Q1	___/1.25	___/1.25	___/1.25	___/0.75	___/0.50	___/5
Q2	___/1.25	___/1.25	___/1.25	___/0.75	___/0.50	___/5
Q3	___/1.25	___/1.25	___/1.25	___/0.75	___/0.50	___/5
Q4	___/1.25	___/1.25	___/1.25	___/0.75	___/0.50	___/5
Q5	___/1.25	___/1.25	___/1.25	___/0.75	___/0.50	___/5
Total	___/6.25	___/6.25	___/6.25	___/3.75	___/2.50	___/25

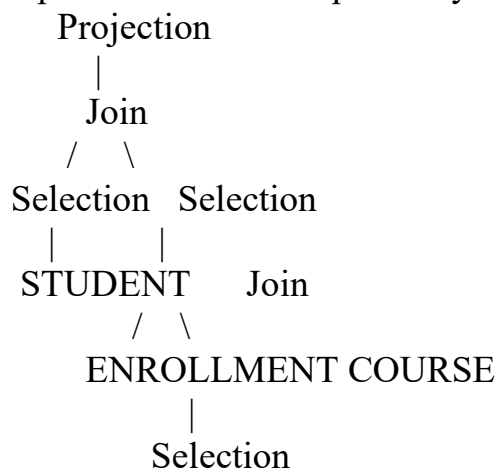
Course Faculty Signature

Q1. For a complex SQL query containing nested joins and sub-queries, query optimization is used to find an efficient way of executing the query with minimum processing cost. The SQL query is first converted into a relational algebra expression, and then a query execution plan or query tree is generated. In the execution plan, the base tables are shown at the bottom, selection and projection operations are placed above them, and join operations are performed at higher levels. For example, consider a query that retrieves student names from the STUDENT table who are enrolled in a course offered by the DEPARTMENT table. The initial query may contain a nested sub-query and multiple joins. The initial execution plan can be represented as:



The heuristic optimization rules are then applied step-by-step. First, perform selections as early as possible. Conditions in the WHERE clause are pushed down toward the base tables so that unwanted tuples are removed before joins. Second, perform projections early by selecting only the required attributes instead of carrying unnecessary columns through the entire query. Third, replace Cartesian products followed by selection with join operations, because a direct join is more efficient. Fourth, reorder joins so that the most restrictive or smaller relations are joined first, reducing the number of intermediate tuples. Finally, unnecessary operations and redundant conditions are removed.

Thus, the optimized execution plan may become:



In this optimized plan, selection operations are performed before joins, reducing the number of tuples involved in subsequent operations. Required attributes are also projected early, reducing memory and disk usage. The final optimized query therefore produces the same result as the original query but with fewer intermediate records and lower execution cost. Heuristic optimization is useful because it improves query performance without examining every possible execution plan, making it suitable for complex SQL queries involving nested joins and sub-queries.

Q2. Cost-based optimization selects the most efficient query execution plan by estimating the cost of different operations. When joining two relations, the three common algorithms are **Nested-Loop Join, Sort-Merge Join, and Hash Join**. The optimizer considers factors such as the **number of tuples, number of pages, available memory, and page size**.

Suppose relation **R** contains **10,000 tuples** occupying **100 pages**, and relation **S** contains **5,000 tuples** occupying **50 pages**. The estimated costs of the three join methods can be compared as follows.

For a **Nested-Loop Join**, the cost depends on the number of pages in the outer and inner relations. Using **R** as the outer relation, the approximate cost is:

$$\text{Cost} = B(R) + B(R) \times B(S) \\ = 100 + (100 \times 50) = 5,100 \text{ page I/Os}$$

If **S** is selected as the outer relation:

$$\text{Cost} = 50 + (50 \times 100) = 5,050 \text{ page I/Os}$$

Therefore, choosing the smaller relation as the outer relation can slightly reduce the cost.

For a **Sort-Merge Join**, both relations must generally be sorted before merging. The cost includes reading, sorting, and merging the relations. If the relations are already sorted on the join attribute, the cost can be significantly lower. Otherwise, sorting introduces additional disk I/O, making the method more expensive than hash join for many equality-join cases.

For a **Hash Join**, a hash function is applied to the join attribute. The relations are divided into hash partitions and matching partitions are then joined. For an equality join, the approximate basic cost is around:

$$\text{Cost} \approx 3 \times (B(R) + B(S))$$

Therefore:

$$\text{Cost} \approx 3 \times (100 + 50) = 450 \text{ page I/Os}$$

This is considerably lower than the nested-loop cost in this example.

Join Method Approximate Cost Suitability

Nested-Loop Join	5,050–5,100 I/Os	Simple, but expensive for large relations
Sort-Merge Join	Depends on sorting cost	Good when relations are already sorted or for ordered/range processing
Hash Join	≈ 450 I/Os	Very efficient for equality joins

Therefore, based on the estimated costs, **Hash Join is the best choice** for this example because it requires significantly fewer page I/Os than Nested-Loop Join and avoids the sorting overhead of Sort-Merge Join. However, the final choice depends on the actual tuple counts, page sizes, available memory, indexes, and type of join condition. **Cost-based optimization chooses the join algorithm with the lowest estimated execution cost**, thereby improving overall query performance.

Q3. import java.sql.*;

```
public class JDBCExample {
    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/universitydb";
        String user = "root";
        String password = "root";

        String query = "SELECT * FROM STUDENT WHERE Department = ?";

        try {
            // Establish connection
            Connection con = DriverManager.getConnection(
                url, user, password);

            // Create PreparedStatement
            PreparedStatement ps = con.prepareStatement(query);

            // Set parameter
            ps.setString(1, "IT");

            // Execute query
            ResultSet rs = ps.executeQuery();

            // Display records
            while (rs.next()) {
                System.out.println(
                    "Student ID: " + rs.getInt("StudentID") +
                    ", Name: " + rs.getString("Name") +
                    ", Department: " + rs.getString("Department")
                );
            }

            // Close resources
            rs.close();
            ps.close();
            con.close();

        } catch (SQLException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Q4. Conflict serializability is a method used in DBMS to determine whether a schedule of concurrent transactions is equivalent to some serial execution of those transactions. It is checked using a **precedence graph**, where each transaction is represented by a node. An edge is added from transaction **T_i** to **T_j** if an operation of T_i conflicts with and occurs before an operation of T_j on the same data item.

Consider the following schedule of four transactions:

T1: R(A) W(B)
 T2: R(A) W(C)
 T3: R(B) W(A)
 T4: R(C) W(B)

Step 1: Identify conflicting operations

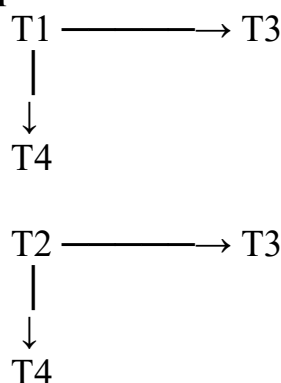
Two operations conflict when:

- They belong to different transactions.
- They access the same data item.
- At least one of them is a **write** operation.

From the schedule:

- T1 W(B) occurs before T3 R(B) → **T1 → T3**
- T1 W(B) occurs before T4 W(B) → **T1 → T4**
- T2 R(A) occurs before T3 W(A) → **T2 → T3**
- T2 W(C) occurs before T4 R(C) → **T2 → T4**
- T3 W(A) occurs after T2 R(A) and before any later conflicting operation on A.

Step 2: Construct the precedence graph



The graph contains **no cycle**.

Step 3: Check for cycles

The important rule is:

- **No cycle** → **Conflict-serializable**
- **Cycle** → **Not conflict-serializable**

Since the precedence graph has no cycle, the given schedule is **conflict-serializable**.

A possible **topological ordering** is:

T1 → T2 → T3 → T4

Therefore, the concurrent schedule is equivalent to the serial schedule:

T1, T2, T3, T4

Conclusion

The precedence graph is used to identify conflicts between concurrent transactions. After creating the graph, if the graph is **acyclic**, the schedule is conflict-serializable. For the above schedule, there is no cycle, so it is **conflict-serializable**, and a valid serial order is **T1 → T2 → T3 → T4**. This technique helps DBMS maintain consistency while allowing multiple transactions to execute concurrently.

Q5. Two-Phase Locking (2PL) is a concurrency-control technique used in DBMS to ensure **conflict serializability**. In Basic 2PL, a transaction has two phases: **Growing Phase** and **Shrinking Phase**. During the growing phase, a transaction can acquire locks but cannot release any lock. During the shrinking phase, it can release locks but cannot acquire any new locks.

Consider two transactions:

T1: Read(A)
Read(B)
Write(A)

T2: Read(B)
Read(A)
Write(B)

Step 1: Apply locks

A **Shared (S) lock** is used for reading, while an **Exclusive (X) lock** is used for writing.

T1: Lock-S(A)
Read(A)
Lock-X(A)
Write(A)
Lock-S(B)
Read(B)
Unlock(A)
Unlock(B)

T2: Lock-S(B)
Read(B)
Lock-X(B)
Write(B)
Lock-S(A)
Read(A)
Unlock(B)
Unlock(A)

The transaction first obtains the required locks during the **growing phase**. Once it starts releasing locks, it enters the **shrinking phase** and cannot obtain any new locks.

Deadlock

A deadlock can occur when two transactions wait for each other's locks.

For example:

T1 → Locks A
T2 → Locks B

T1 → Requests B → waits for T2

T2 → Requests A → waits for T1

This creates a **circular wait**:

T1 → T2
↑ ↓
└─────┘

Therefore, **T1 and T2 are in deadlock.**

Deadlock Resolution

The DBMS can resolve the deadlock by:

1. **Detecting the cycle** in the wait-for graph.
2. Selecting one transaction as the **victim**.
3. Rolling back the selected transaction.
4. Releasing its locks.
5. Allowing the remaining transaction to continue.
6. Restarting the rolled-back transaction later.

For example, if **T2 is selected as the victim**, T2 is rolled back and its locks are released. T1 can then acquire the required lock on B and complete its execution. T2 can subsequently be restarted.

Conclusion

Basic 2PL ensures that transactions follow a proper locking protocol and helps achieve **conflict serializability**. However, it can cause deadlocks when transactions wait for each other's locks. Deadlocks can be detected using a **wait-for graph** and resolved by aborting or rolling back one of the transactions. Thus, 2PL provides consistency and safe concurrent execution in a DBMS.